Game Summary: We will make a Pac-Man-inspired game in which the user can control Pac-Man using a keyboard and move him within the bounds of a maze and the screen. The user can collect coins by touching them to make them disappear. There will be three coins in hard-coded locations and a scoring system on HEX0. We will add a ghost, and display the score on the VGA display if time permits.

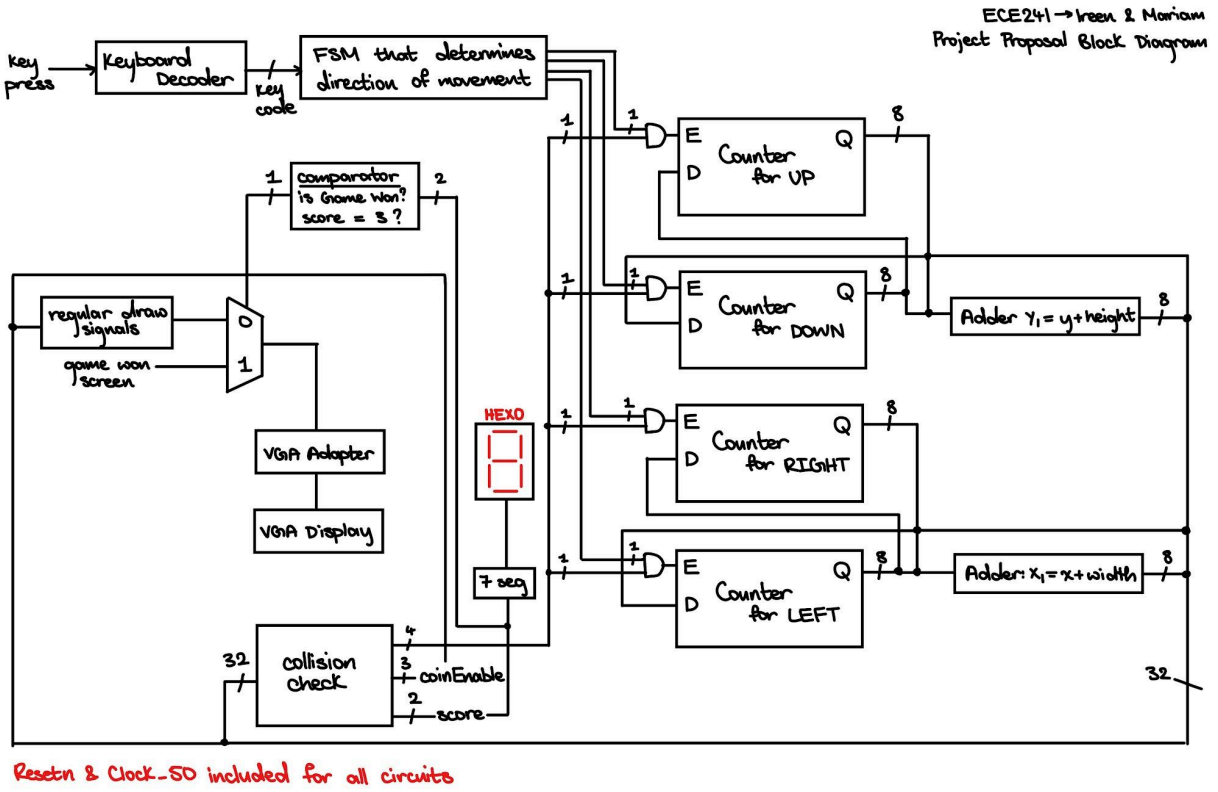


Figure 1. Block Diagram

Table 1. Project Schedule

Day	Week One (Nov 3rd)	Week Two (Nov 10th)	Week Three (Nov 17th)
Tues	Project Proposal	Collision Checking	
Sat	Draw Maze, Pacman, Coins		Extra Game Logic
Sun	Make Pacman Move in All Directions	Keyboard & Scoring System + Game Won	Presentation Preparation
Mon	Milestone → Basic VGA Display & Pac-Man Animations	Milestone → VGA Display with Pac-Man Animations	Milestone → Project Presentation

Division of Work (Referencing Block Diagram):

Mariam → All Counters, Both Adders, Collision Check with Coins, VGA Adapter, Scoring System

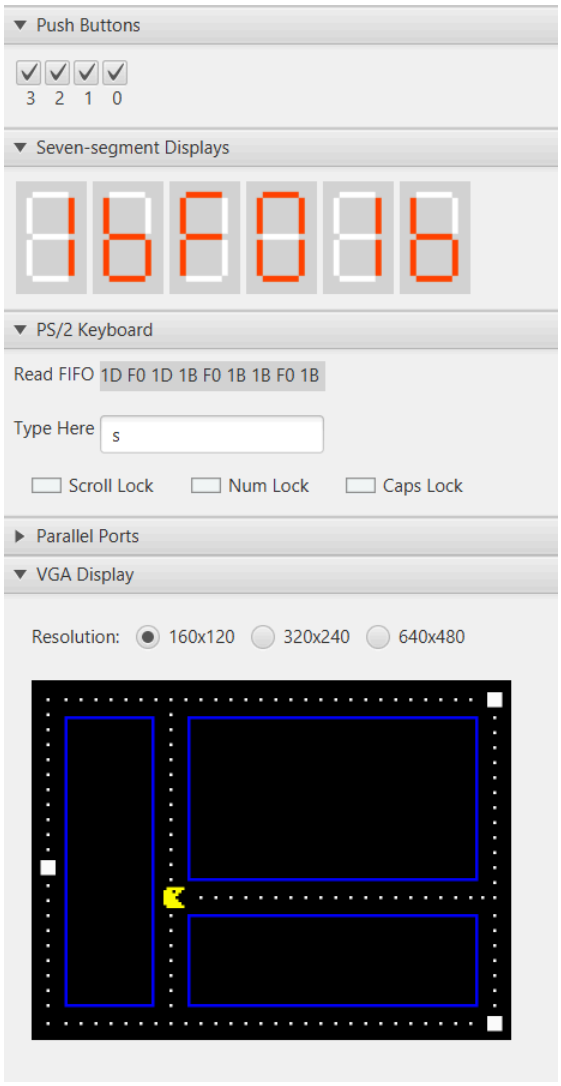
Ireen → Collision Check with Maze, FSM for Movement, FPGA Keys, PS/2 Keyboard, Pixel Art

Division of Work - MILESTONE 1

Mariam	Ireen
- Code for checking collision with coins and maze walls, including updating score and movement enable - Writing ModelSim testbench to simulate code - Created counters to move Pacman's location, but this was scrapped after we realized the demo code already provided this. We will re-use this code for other features (i.e the ghost)	- Pixel art for Pacman, Coins, Maze - modifying demo code to draw moveable objects and static background to screen - Writing additional code to make Pacman's movement scroll smoothly (i.e when Pacman moves off screen, he should appear on the other side of the screen in the next movement)

Code for Demo

Drawing Pacman and Other Objects to VGA Display



1. In vga_demo Module

```
// The signals below are used to multiplex the pixels displayed for objects 1 and 2
//Ireen: Added two more parameters for two more objects O3 and O4 on each line
wire [nX-1:0] O1_x, O2_x, O3_x, O4_x, MUX_x;     // x coordinate multiplexer
wire [nY-1:0] O1_y, O2_y, O3_y, O4_y, MUX_y;     // y coordinate multiplexer
wire [COLOR_DEPTH-1:0] O1_color, O2_color, O3_color, O4_color, MUX_color; // color multiplexer
wire O1_write, O2_write, O3_write, O4_write, MUX_write; // write control multiplexer
```

2. In vga_demo Module

```
//Ireen: added two more object dones O3 and O4
wire O1_done, O2_done, O3_done, O4_done, done;             // object move completed
//Ireen: changed the object selector to a 4bit wire
wire [3:0] O1_O2_O3_O4;          // draw object 1 when cleared, other objects every clock cycle
wire [1:0] O1_dir, O2_dir;              // used to set direction of moving for objects
reg [1:0] y_Q, Y_D;             // FSM, used to control drawing of objects

wire Resetn, KEY1, KEY2;        // Reset, and synchronized versions of KEYs
wire PS2_CLK_S, PS2_DAT_S;      // synchronized versions of PS2 signals

//Ireen: permanently set object select to one to always draw pacman
assign object_sel = 1'b1;
```

3. In vga_demo module

```
  //Ireen: shift register for all objects to constantly be drawn
 shiftRegister SR (4'b1000, KEY[0], 1'b1, 1'b1, CLOCK_50, O1_O2_O3_O4);

 //Ireen: deleted scancode for d f r c keys (not needed)
 assign O1_dir = scancode[1:0]; // ps2 key identifier (for a, s, w, d)
```

4. In vga_demo Module

```
  // instantiate object 1
  //Ireen: changed the object to pacman, set the XOFFSET and YOFFSET parameters
object O1 (Resetn, CLOCK_50, KEY1, object_sel & step, O1_dir, O1_x, O1_y,
            O1_color, O1_write, O1_done);
    defparam O1.RESOLUTION = RESOLUTION;
    defparam O1.LEFT  = 2'b00;  // 'a'
    defparam O1.RIGHT = 2'b11;  // 'd'
    defparam O1.UP    = 2'b01;  // 'w'
    defparam O1.DOWN =  2'b10;  // 's'
    defparam O1.XOFFSET = 47;
    defparam O1.YOFFSET = 73;
    defparam O1.nX = nX;
    defparam O1.nY = nY;
    defparam O1.COLOR_DEPTH = COLOR_DEPTH;
    defparam O1.INIT_FILE = "./MIF/pacman.mif";

    //Ireen: a copy of the above object with changes to draw the coin (still being debugged)
object O2 (Resetn, CLOCK_50, KEY2, !object_sel & step, O2_dir, O2_x, O2_y,
            O2_color, O2_write, O2_done);
    defparam O2.RESOLUTION = RESOLUTION;
    defparam O2.LEFT  = 2'b00;  // 'a'
    defparam O2.RIGHT = 2'b11;  // 'd'
    defparam O2.UP    = 2'b01;  // 'w'
    defparam O2.DOWN =  2'b10;  // 's'
    defparam O2.XOFFSET = 4;
    defparam O2.YOFFSET = 61;
    defparam O2.nX = nX;
    defparam O2.nY = nY;
    defparam O2.COLOR_DEPTH = COLOR_DEPTH;
    defparam O2.INIT_FILE = "./MIF/coin.mif";

    //Ireen: added two more objects O3 and O4
    assign done = O1_done | O2_done | O3_done | O4_done;
```

5. In vga_demo module

```
 //Ireen: multiple assign statements depending on which number the shift register was currently outputting
 //Ireen: these assign statements decide which object (1, 2, 3, or 4) get drawn
   assign MUX_x =
    (O1_O2_O3_O4 == 4'b1000) ? O1_x :
    (O1_O2_O3_O4 == 4'b0100) ? O2_x :
    (O1_O2_O3_O4 == 4'b0010) ? O3_x :
    (O1_O2_O3_O4 == 4'b0001) ? O4_x : 0;

   assign MUX_y =
    (O1_O2_O3_O4 == 4'b1000) ? O1_y :
    (O1_O2_O3_O4 == 4'b0100) ? O2_y :
    (O1_O2_O3_O4 == 4'b0010) ? O3_y :
    (O1_O2_O3_O4 == 4'b0001) ? O4_y : 0;

   assign MUX_color =
    (O1_O2_O3_O4 == 4'b1000) ? O1_color :
    (O1_O2_O3_O4 == 4'b0100) ? O2_color :
    (O1_O2_O3_O4 == 4'b0010) ? O3_color :
    (O1_O2_O3_O4 == 4'b0001) ? O4_color : 0;

   assign MUX_write =
    (O1_O2_O3_O4 == 4'b1000) ? O1_write :
    (O1_O2_O3_O4 == 4'b0100) ? O2_write :
    (O1_O2_O3_O4 == 4'b0010) ? O3_write :
    (O1_O2_O3_O4 == 4'b0001) ? O4_write : 1'b0;
//all above assign statements written by Ireen
```

6. In vga_demo module

```
//Ireen: new module for shift register for O1_O2_O3_O4
module shiftRegister (R, L, E, w, Clock, Q);
   input wire [3:0] R;
   input wire L, w, Clock, E;
   output reg [3:0]Q;

   always @ (posedge Clock)
      if (L == 1'b1)
         Q <= R;
      else if (E == 1'b1)
      begin
         Q[3] <= w;
         Q[2] <= Q[3];
         Q[1] <= Q[2];
         Q[0] <= Q[1];
      end
endmodule
```

```
  // instantiate the DESim VGA adapter. It first draws the background image that is stored
  // in the video memory (the video memory is optional), and then draws pixels for the
  // selected object whenever MUX_write is asserted.
    `define VGA_MEMORY
    vga_adapter VGA (
       .resetn(Resetn),
       .clock(CLOCK_50),
       .color(MUX_color),      // used to draw pixels on top of background
       .x(MUX_x),              // used to draw pixels on top of background
       .y(MUX_y),              // used to draw pixels on top of background
       .write(MUX_write),      // used to draw pixels on top of background
       .VGA_X(VGA_X),          // the output VGA x coordinate (column)
       .VGA_Y(VGA_Y),          // the output VGA y coordinate (row)
       .VGA_COLOR(VGA_COLOR),  // the output VGA color
       .VGA_SYNC(VGA_SYNC),    // indicates when background MIF has been drawn
       .plot(plot));           // the output VGA plot signal
       defparam VGA.RESOLUTION = RESOLUTION;
      // choose background image according to resolution and color depth
       defparam VGA.BACKGROUND_IMAGE = "./MIF/maze_with_coins.mif" ; //Ireen: change background image to maze
       defparam VGA.COLOR_DEPTH = COLOR_DEPTH;
       assign LEDR = {VGA_SYNC,1'b0,scancode}; // turn on LED 9 when background is done, show scancode
```

7. Shift register module

8. Up and down counter module

```
// n-bit up/down-counter with reset, load, enable, and direction control
module upDn_count (R, Clock, Resetn, L, E, Dir, Res, Q);
    //same inputs as the demo updown counter
    parameter n = 8;
    input wire [7:0] Res; //Ireen: New Res input
    input wire [n-1:0] R;
    input wire Clock, Resetn, E, L, Dir;
    output reg [n-1:0] Q;

    //Ireen: Everything written below in this module is new
    //this counter handles pacman moving past the screen edges
    reg [7:0] next_Q;

    always @(*)
    begin
        next_Q = Q;  //default

        if (E)
        begin
            if (Dir)
            begin
            //moving down or right
                if (Q == 8'd163 & Res == 8'd160)
                    next_Q = 8'd253;        //wrap around to start
                else if (Q == 8'd122 & Res == 8'd120)
                    next_Q = 8'd253;
                else
                    next_Q = Q + 1'b1;
            end
            else
            begin
            //moving up or left
                if (Q == 8'd253 & Res == 8'd160)
                    next_Q = Res + 3;      //wrap around to end
                else if (Q == 8'd252 & Res == 8'd120)
                    next_Q = 8'd122;
                else
                    next_Q = Q - 1'b1;
             end
        end
    end

   always @(posedge Clock or negedge Resetn) begin
      if (!Resetn)
         Q <= 8'd0;
      else if (L)
         Q <= R;
      else
         Q <= next_Q;
   end
endmodule
```

9. In object module

```
//Ireen: set object size to 3 because pacman is a 8x8 mif
parameter xOBJ = 3, yOBJ = 3;   // object size is 2^xOBJ x 2^yOBJ
```

10. In object module

```
//Ireen: deleted assignment of parameter for object file
parameter INIT_FILE;
```

Smooth Scrolling of Pacman's Movement

The original pacman object is set to move with respect to its origin/center. The original object vga_demo file provides code that allows any drawn object to move past the edges of the screen and consequently show up on the other respective side of the screen (e.g. moving past the right side of the screen will draw the object on the left side after a long time). However, this was not optimal as the coordinates would have to reach 255 on either the X or Y axis in order to show up on the other side. To fix this, we implemented new code inside the upDn counter module to update the position of the object when it reaches the screen edges. It does not check only the origin, it checks to make sure the entire object is past the screen before drawing it on the other side.

```
1
2  //Based on upDn_count from the object VGA demo on Quercus
3  //Module definition and ports are the same as the original
4  module upDn_count (R, Clock, Resetn, L, E, Dir, Res, Q);
5      parameter n = 8;
6      input wire [7:0] Res; //Resolution of the screen in the direction we are moving in (160 == xDir, 120 == yDir)
7      input wire [n-1:0] R; //Initial position
8      input wire Clock, Resetn, E, L, Dir;
9      output reg [n-1:0] Q;
10
11     //Beyond this point is modified from the demo
12     //nextQ is added by us
13     reg [7:0] next_Q;
14
15     always @(*)
16     begin
17         next_Q = Q;  // default
18
19         //Checking for whether scrolling is needed is added by us.
20         //The default cases (increment Q and decrement Q and Dir) are from the original code which we modified
21
22         if (E) //While E = 1, the object is moving
23          begin
24             if (Dir) //While Dir = 1, object is moving right or down
25             begin
26                 // moving down or right
27                 //Check if moving right and object is past the right edge of the screen
28                 if (Q == 8'd163 & Res == 8'd160)
29                     next_Q = 8'd253;       // wrap around to start
30
31                 //Check if moving down and object is past the bottom edge of the screen
32                 else if (Q == 8'd122 & Res == 8'd120)
33                     next_Q = 8'd253;
34                 else
35                 //Increment position like usual
36                     next_Q = Q + 1'b1;
37             end
38             else //While Dir = 0, the object is moving left or up
39             begin
40                 // moving up or left
41                 //Check if moving left and past the left edge of the screen
42                 if (Q == 8'd253 & Res == 8'd160)
43                     next_Q = Res + 3;     // wrap around to end
44
45                 //Check if moving up and past the top edge of the screen
46                 else if (Q == 8'd252 & Res == 8'd120)
47                     next_Q = 8'd122;
48                 else
49                 //Decrement position like usual
50                     next_Q = Q - 1'b1;
51             end
52          end
53     end
54
55     //From demo code
56     always @(posedge Clock or negedge Resetn) begin
57         if (!Resetn)
58             Q <= 8'd0;
59         else if (L)
60             Q <= R;
61         else
62             Q <= next_Q;
63     end
64 endmodule
```

ModelSim

Summary of test input

- Reset circuit

- Teleport object to be just outside the right edge of the screen
- Let the object continue moving right. It will reach 0 before 255.
- Teleport object to be just outside the left edge of the screen.
- Let the object continue moving left. It will teleport to the right edge of the screen.



Checking Collision Between Pacman and Objects

This code works by checking collisions between Pacman and other objects (such as the coins and maze walls) using a module. The coordinates of Pacman and other objects are taken using the origin as the top-left corner. Because a lot of boolean statements are needed, we used modules to evaluate the statements for readability. Also because it allows us to only write each statement once, so if we need to make changes (which we had to do during debugging), we only have to make the change in one place and not three places. The coordinates of Pacman are passed in as input and the coordinates of the maze walls and coins are hard-coded using parameters. The program changes based on the collision every clock cycle, using an always block. On reset, the coins are all set to be enabled and the score is set to 0. Movement in any direction is disabled.

```
module collision_checker(Clock, Resetn, pacmanX, pacmanX_1, pacmanY, pacmanY_1, coinEnable, moveEnable, score);
    input Clock, Resetn;
    input [7:0] pacmanX, pacmanX_1, pacmanY, pacmanY_1; //Represents pacman's coordinates (xPos, xPos + width, yPos, yI

    output reg [2:0] coinEnable; //Whether each coin1, coin2, coin3 should be drawn to the screen
    output reg [1:0] score;

    output reg [3:0] moveEnable; //0 = up, 1 = down, 2 = left, 3 = right

    //parameters representing coin location
    parameter coin1X = 8'd3, coin1X_1 = 8'd8, coin1Y = 8'd60, coin1Y_1 = 8'd65, coin2X = 8'd152, coin2X_1 = 8'd157, co
              coin3X = 8'd152, coin3X_1 = 8'd157, coin3Y = 8'd112, coin3Y_1 = 8'd117;

    //parameters representing maze wall location
    parameter maze1X = 8'd11, maze1X_1 = 8'd40, maze1Y = 8'd12, maze1Y_1 = 8'd108, maze2X = 8'd52, maze2X_1 = 8'd148, m
              maze3X = 8'd52, maze3X_1 = 8'd148, maze3Y = 8'd78, maze3Y_1 = 8'd108;

    wire colCoin1, colCoin2, colCoin3;

    coin_collision_checker U1(pacmanX, pacmanX_1, pacmanY, pacmanY_1, coin1X, coin1X_1, coin1Y, coin1Y_1, colCoin1);
    coin_collision_checker U2(pacmanX, pacmanX_1, pacmanY, pacmanY_1, coin2X, coin2X_1, coin2Y, coin2Y_1, colCoin2);
    coin_collision_checker U3(pacmanX, pacmanX_1, pacmanY, pacmanY_1, coin3X, coin3X_1, coin3Y, coin3Y_1, colCoin3);

    wire [3:0] maze1Col, maze2Col, maze3Col;

    maze_left_collision U4(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze1X, maze1X_1, maze1Y, maze1Y_1, maze1Col[2]);
    maze_left_collision U5(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze2X, maze2X_1, maze2Y, maze2Y_1, maze2Col[2]);
    maze_left_collision U6(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze3X, maze3X_1, maze3Y, maze3Y_1, maze3Col[2]);

    maze_right_collision U7(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze1X, maze1X_1, maze1Y, maze1Y_1, maze1Col[3]);
    maze_right_collision U8(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze2X, maze2X_1, maze2Y, maze2Y_1, maze2Col[3]);
    maze_right_collision U9(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze3X, maze3X_1, maze3Y, maze3Y_1, maze3Col[3]);

    maze_top_collision U10(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze1X, maze1X_1, maze1Y, maze1Y_1, maze1Col[0]);
    maze_top_collision U11(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze2X, maze2X_1, maze2Y, maze2Y_1, maze2Col[0]);
    maze_top_collision U12(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze3X, maze3X_1, maze3Y, maze3Y_1, maze3Col[0]);

    maze_bottom_collision U13(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze1X, maze1X_1, maze1Y, maze1Y_1, maze1Col[1])
    maze_bottom_collision U14(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze2X, maze2X_1, maze2Y, maze2Y_1, maze2Col[1])
    maze_bottom_collision U15(pacmanX, pacmanX_1, pacmanY, pacmanY_1, maze3X, maze3X_1, maze3Y, maze3Y_1, maze3Col[1])


    //Check collision with pacman and coins, maze walls
    always@(posedge Clock)
    begin

        //Reset
        if (!Resetn)
            begin
                //Set default values
                coinEnable <= 3'b111;
                score <= 2'b0;
                moveEnable <= 4'b0000;
            end
        else
        begin
        //Coin 1
        if (colCoin1)
            if (coinEnable[0])
            begin
                score <= score + 1;
                coinEnable[0] <= 0;
            end

        //Coin 2
        if (colCoin2)
            if (coinEnable[1])
            begin
                score <= score + 1;
                coinEnable[1] <= 0;
            end

        //Coin 3
        if (colCoin3)
            if (coinEnable[2])
            begin
                score <= score + 1;
                coinEnable[2] <= 0;
            end


        //Assume Pacman moves at a speed of 1 px
        //Set enable for movement based on collision with maze walls

        //Top
        moveEnable[0] <= ~(maze1Col[0] | maze2Col[0] | maze3Col[0]);
        //Bottom
        moveEnable[1] <= ~(maze1Col[1] | maze2Col[1] | maze3Col[1]);
        //Left side
        moveEnable[2] <= ~(maze1Col[2] | maze2Col[2] | maze3Col[2]);
        //right side
        moveEnable[3] <= ~(maze1Col[3] | maze2Col[3] | maze3Col[3]);
        end
    end

endmodule
```

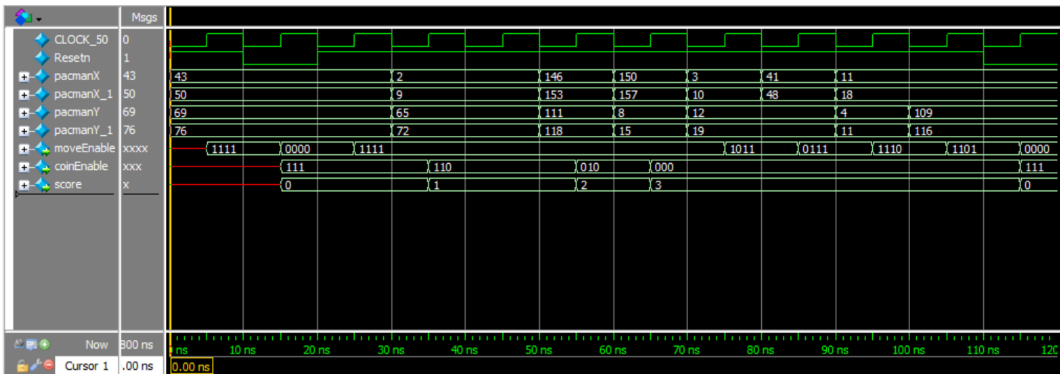
```
98  //This module does the if statement for the collision with a coin. Result = 1 when colliding and 0 otherwise
99  module coin_collision_checker(pacmanX, pacmanX_1, pacmanY, pacmanY_1,
100    coinX, coinX_1, coinY, coinY_1, result);
101
102    input [7:0] pacmanX, pacmanX_1, pacmanY, pacmanY_1, coinX, coinX_1, coinY, coinY_1;
103    output result;
104    wire collidesLeft, collidesRight, collidesUp, collidesDown;
105
106    assign collidesLeft = (coinX <= pacmanX_1) & (coinX >= pacmanX) & (pacmanY_1 >= coinY) & (pacmanY <= coinY_1);
107
108    assign collidesRight = (coinX >= pacmanX) & (coinX <= pacmanX_1) & (pacmanY_1 >= coinY) & (pacmanY <= coinY_1);
109
110    assign collidesDown = (coinY_1 >= pacmanY) & (coinY <= pacmanY) & (pacmanX_1 >= coinX) & (pacmanX <= coinX_1);
111
112    assign collidesUp = (pacmanY_1 >= coinY) & (pacmanY <= coinY_1) & (pacmanX_1 >= coinX) & (pacmanX <= coinX_1);
113
114    assign result = collidesLeft | collidesRight | collidesUp | collidesDown;
115
116 endmodule
117
118 //This module outputs 1 if pacman collides with the LEFT side of a maze wall
119 module maze_left_collision(pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1, result);
120    input [7:0] pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1;
121    output result;
122
123    assign result = (pacmanX_1 == mazeX - 1) & (pacmanY_1 >= mazeY) & (pacmanY <= mazeY_1);
124 endmodule
125
126 //This module outputs 1 if pacman collides with the RIGHT side of a maze wall
127 module maze_right_collision(pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1, result);
128    input [7:0] pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1;
129    output result;
```

```
134 //This module outputs 1 if pacman collides with the TOP side of a maze wall
135 module maze_top_collision(pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1, result);
136    input [7:0] pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1;
137    output result;
138
139    assign result = (pacmanY_1 == mazeY - 1) & (pacmanX_1 >= mazeX) & (pacmanX <= mazeX_1);
140 endmodule
141
142 //This module outputs 1 if pacman collides with the BOTTOm side of a maze wall
143 module maze_bottom_collision(pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1, result);
144    input [7:0] pacmanX, pacmanX_1, pacmanY, pacmanY_1, mazeX, mazeX_1, mazeY, mazeY_1;
145    output result;
146
147    assign result = (pacmanY == mazeY_1 + 1) & (pacmanX_1 >= mazeX) & (pacmanX <= mazeX_1);
148 endmodule
```

ModelSim

Summary of test input:

- Reset for one clock cycle
- Test if Pacman will collide with coin 1 (its coordinates are set so that it overlaps with coin 1 from below)
- Leave Pacman in this position for an additional clock cycle. The score should not increase past 1.
- Move Pacman to overlap with coin 2.
- Move Pacman to overlap with coin 3.
- Move Pacman to touch the edges of Maze Wall 1 (left edge, right edge, top edge, bottom edges)
- Reset



Division of Work - MILESTONE 2

Mariam	Ireen
- Wrote new coin collision logic - Wrote ghost logic and ghost collision - Drew Game Won screen and ghost - Displayed score on the hex display	- Implemented coin and maze collision logic with the game itself - Made the coins disappear immediately when collided with - Made the game won screen appear after all coins were collected - Implemented keyboard so that pacman only moves using WASD and no other keys

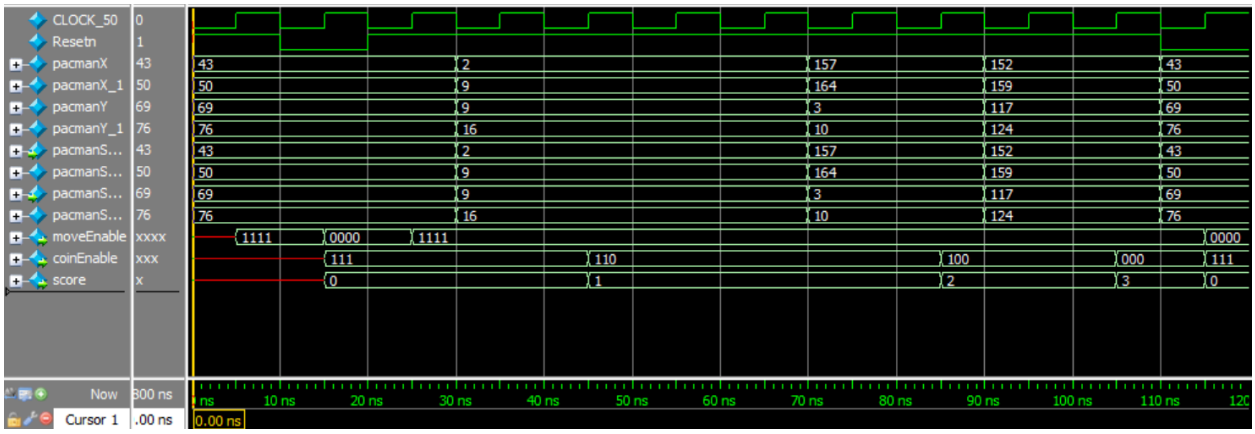
Collision Checker

We modified the coin collision logic from the milestone 1 collision logic as we noticed that the logic did not work correctly in some cases on the board. The logic has been changed to be more in-line with the working maze collision logic. It now only checks if Pacman is touching the edge of a coin rather than overlapping with a coin (this change is because it made it easier to make the coin disappear from the screen).

```
117  //This module does the if statement for the collision with a coin. Result = 1 when colliding and 0 otherwise
118  module coin_collision_checker(pacmanX, pacmanX_1, pacmanY, pacmanY_1, coinX, coinX_1, coinY, coinY_1, result);
119
120     input [7:0] pacmanX, pacmanX_1, coinX, coinX_1;
121     input [6:0]pacmanY, pacmanY_1, coinY, coinY_1;
122     output result;
123     wire collidesLeft, collidesRight, collidesUp, collidesDown;
124
125     assign collidesRight = (coinX_1 + 1'b1 == pacmanX) & (pacmanY_1 >= coinY) & (pacmanY <= coinY_1);
126     assign collidesLeft = (coinX - 1'b1 == pacmanX_1) & (pacmanY_1 >= coinY) & (pacmanY <= coinY_1);
127     assign collidesUp = (coinY_1 + 1'b1 == pacmanY) & (pacmanX <= coinX_1) & (pacmanX_1 >= coinX);
128     assign collidesDown = (coinY - 1'b1 == pacmanY_1) & (pacmanX <= coinX_1) & (pacmanX_1 >= coinX);
129
130     assign result = collidesLeft | collidesRight | collidesUp | collidesDown;
131
132  endmodule
```

The ModelSim simulation below has the following tests on the collision logic:

- Begin with from Reset, with Pacman in his default location
- Move Pacman so that he collides with coin 1. The score should increase after one clock cycle.
- Leave Pacman in this position for a few more clock cycles. The score should not increase further.
- Move Pacman so that he collides with coin 2. The score should increase after one clock cycle.
- Move Pacman so that he collides with coin 3. The score should increase after one clock cycle.
- Reset the board. All coins should be enabled again and the score should be 0.
- Pacman's movement should remain enabled across all test cases because he is not touching the maze walls in any of these positions.



Pulse Module for Coin Collisions

We noticed in testing that objects were only able to appear when we connected the signal that should draw them to a key press. To make objects appear automatically, we created this pulse module to simulate a key press.

```
//Ireen: implemented pulse module that emulates a key press to replace KEY1, KEY2, KEY3
module pulse (Clock, Resetn, Enable, Q);
   input wire Clock, Resetn, Enable;
   output wire Q;
   reg out, prevEnable;

   always @ (posedge Clock)
   begin
      if (!Resetn)
      begin
         prevEnable <= 1'b0;
         out <= 1'b0;
      end
      else
      begin
         out <= Enable & ~prevEnable;
         prevEnable <= Enable;
      end
   end
   assign Q = out;
endmodule
```

Coin/Maze Collision and Movement/Keyboard Logic

This code contains the implementation of the collision logic from milestone 1 into the main module that is used for the board and VGA. The stableX and stableY signals were added because the code would not work properly on the board if Pacman's X and Y coordinates were used directly. Their value is always the same as Pacman's X and Y coordinates, however. The code instantiates the collision checker module and creates wires for the necessary inputs and outputs. It connects the outputs to the signals that are used for drawing and moving objects, such as Pacman. To make the coins disappear from the screen, we draw a black square over the coin on the screen. There is logic to decide which object should be drawn (choosing between Pacman, black square over each coin, and the game won screen). We found that the game won screen would not draw correctly if we tried to draw it at the same time as the black square over the final coin to be collected. Therefore, we use the isGameWon signal to

delay the drawing of the game won screen. After the game won screen is drawn, it is set to always be the selected object to be drawn, preventing the screen from changing further until reset.

```
//Ireen: select pacman to draw
reg select;
assign object_sel = select;

reg [2:0]temp_direction;
wire [2:0] coinEnable;
wire [2:0] coinDisable;
wire [3:0] moveEnable;
wire [1:0] score;

wire [7:0] stableX;
wire [6:0] stableY;

collision_checker U1(CLOCK_50, Resetn, pacman_x, pacman_y, coinEnable, moveEnable, score, stableX, stableY);
//Ireen: variables to indicate if you won the game
reg isGameWon;
reg gameWon;

always @ (*)
   if (score == 2'b11) //Ireen: if score is 3, game is won, pacman can't move
   begin
      isGameWon = 1'b1;
      select = 1'b0;
   end
   else //Ireen: if score is not 3 then game is not won and pacman can move
   begin
      isGameWon = 1'b0;
      select = 1'b1;
   end

always @(posedge CLOCK_50)
begin
      if (!Resetn)
         gameWon <= 1'b0;
      else if (isGameWon)
         gameWon <= 1'b1;   //Ireen: indicate that the game should end
end

pulse P1 (CLOCK_50, Resetn, !coinEnable[0], coinDisable[0]); //Ireen: emulates a key press, coinDisable is 1 for 1 clock cycle
pulse P2 (CLOCK_50, Resetn, !coinEnable[1], coinDisable[1]);
pulse P3 (CLOCK_50, Resetn, !coinEnable[2], coinDisable[2]);

always @ (*)
   case (scancode[7:0])//Ireen: for moveEnable: 0 = up, 1 = down, 2 = left, 3 = right
       7'h1d: if (moveEnable[0]) temp_direction <= 3'b001; else temp_direction <= 3'b100;
       7'h1b: if (moveEnable[1]) temp_direction <= 3'b010; else temp_direction <= 3'b100;
       7'h1c: if (moveEnable[2]) temp_direction <= 3'b000; else temp_direction <= 3'b100;
       7'h23: if (moveEnable[3]) temp_direction <= 3'b011; else temp_direction <= 3'b100;
       //Ireen: for temp_direction: LEFT = 3'b000 /*'d'*/, RIGHT = 3'b011/*'a'*/, UP = 3'b001/*'w'*/, DOWN = 3'b010/*'s'*/
       default: temp_direction <= 3'b100; //Ireen: don't move
   endcase

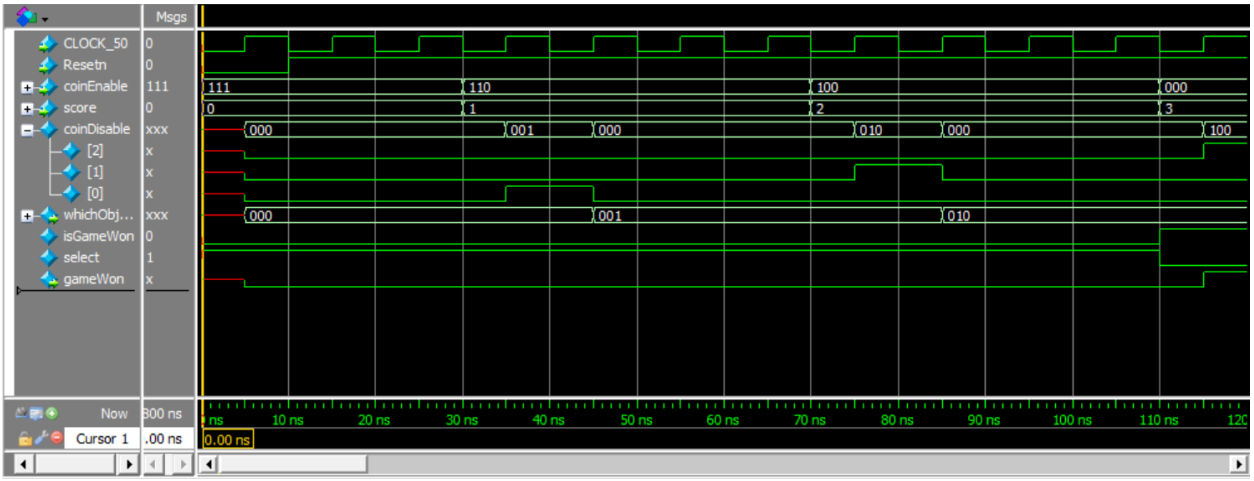
//Ireen: pacman's direction of movement based on the keys pressed
assign pacman_dir = temp_direction;

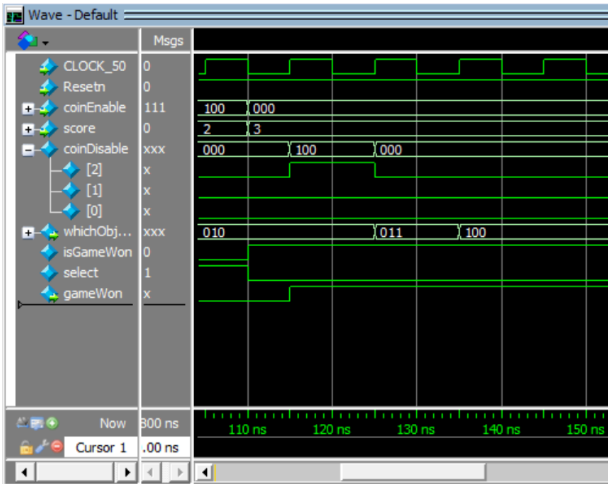
//Ireen: chooses which object to select based on the game logic/conditions
always @(posedge CLOCK_50)
   if (!Resetn | KEY0)
      whichObject <= 3'b000;  // select pacman on reset
   else if (coinDisable[0])
      whichObject <= 3'b001;  // select coin1 if collided with
   else if (coinDisable[1])
      whichObject <= 3'b010;  // select coin2 if collided with
   else if (coinDisable[2])
      whichObject <= 3'b011;  // select coin3 if collided with
   else if (step & (gameWon == 0))
      whichObject <= 3'b000;  // select pacman again after collision with a coin if game has not been won
   else if (gameWon & (coin1_done | coin2_done | coin3_done))
      whichObject <= 3'b100;  // draw game won screen when all coins are collected and have disappeared
```

The below ModelSim was done with a modified version of the above code (and is used only to test that the signals for drawing the black squares and the game won screen works correctly. The applicable part of the code was put in its own module specifically for ModelSim testing. The below simulation shows the following tests:

- Reset the board.
- Collect coin 1 (coinEnable[0]). There should be a short pulse from coinDisable[0], which allows a black square to be drawn over coin 1.

- Collect coin 2 (coinEnable[1]). There should be a short pulse from coinDisable[1], which allows a black square to be drawn over coin 2.
- Collect coin 3 (coinEnable[2]). There should be a short pulse from coinDisable[2], which allows a black square to be drawn over coin 2. Additionally, isGameWon should be set to 1 and then gameWon should be set to 1 on the next positive edge of the clock.
- whichObject should update its value to reflect which object is currently being drawn. Note that in the simulation, whichObject will not be set back to Pacman after the coin fully disappears. This is because our code does this when the player presses one of the movement keys, but in the simulation, we assumed none of the movement keys are being pressed.
    - 000: Pacman
    - 001: black square over coin 1
    - 010: black square over coin 2
    - 011: black square over coin 3.
    - 100: game won screen





The code below creates the five objects used for drawing images: pacman, the black coins, and the game won screen.
Disabled the movement of objects that weren't pacman by setting direction to 3'b100 and object_sel to 0

```
//dummy variables that won't be used
wire [7:0] dummyStableX;
wire [6:0] dummyStableY;

 //Ireen: created pacman, three black coins (for disappearance) and the game won screen
object O1 (Resetn, CLOCK_50, KEY0, object_sel & step, pacman_dir, pacman_x, pacman_y,
            pacman_color, pacman_write, pacman_done, stableX, stableY);
   defparam O1.RESOLUTION = RESOLUTION;
   defparam O1.LEFT  = 2'b00;  // 'a'
   defparam O1.RIGHT = 2'b11;  // 'd'
   defparam O1.UP    = 2'b01;  // 'w'
   defparam O1.DOWN =  2'b10;  // 's'
   defparam O1.XOFFSET = 47;
   defparam O1.YOFFSET = 73;
   defparam O1.nX = nX;
   defparam O1.nY = nY;
   defparam O1.COLOR_DEPTH = COLOR_DEPTH;
   defparam O1.INIT_FILE = "./MIF/pacman.mif";

//top left coin
object O2 (Resetn, CLOCK_50, coinDisable[0], 1'b0, 3'b100, coin1_x, coin1_y,
            coin1_color, coin1_write, coin1_done, dummyStableX, dummyStableY);
   defparam O2.RESOLUTION = RESOLUTION;
   defparam O2.XOFFSET = 4;
   defparam O2.YOFFSET = 5;
   defparam O2.nX = nX;
   defparam O2.nY = nY;
   defparam O2.COLOR_DEPTH = COLOR_DEPTH;
   defparam O2.INIT_FILE = "./MIF/black_coin.mif";

//top right coin
object O3 (Resetn, CLOCK_50, coinDisable[1], 1'b0, 3'b100, coin2_x, coin2_y,
            coin2_color, coin2_write, coin2_done, dummyStableX, dummyStableY);
   defparam O3.RESOLUTION = RESOLUTION;
   defparam O3.XOFFSET = 156;
   defparam O3.YOFFSET = 5;
   defparam O3.nX = nX;
   defparam O3.nY = nY;
   defparam O3.COLOR_DEPTH = COLOR_DEPTH;
   defparam O3.INIT_FILE = "./MIF/black_coin.mif";

//bottom right coin
object O4 (Resetn, CLOCK_50, coinDisable[2], 1'b0, 3'b100, coin3_x, coin3_y,
            coin3_color, coin3_write, coin3_done, dummyStableX, dummyStableY);
   defparam O4.RESOLUTION = RESOLUTION;
   defparam O4.XOFFSET = 156;
   defparam O4.YOFFSET = 116;
   defparam O4.nX = nX;
   defparam O4.nY = nY;
   defparam O4.COLOR_DEPTH = COLOR_DEPTH;
   defparam O4.INIT_FILE = "./MIF/black_coin.mif";


object O5 (Resetn, CLOCK_50, gameWon & (coin1_done | coin2_done | coin3_done), 1'b0, 3'b100, gameWon_x, gameWon_y,
           gameWon_color, gameWon_write, gameWon_done, dummyStableX, dummyStableY);
  defparam O5.RESOLUTION = RESOLUTION;
  defparam O5.xOBJ = 7;
  defparam O5.yOBJ = 6;
  defparam O5.nX = nX;
  defparam O5.nY = nY;
  defparam O5.COLOR_DEPTH = COLOR_DEPTH;
  defparam O5.INIT_FILE = "./MIF/gameWon.mif";

  assign done = pacman_done | coin1_done | coin2_done | coin3_done | gameWon_done | gameOver_done;
```

Pick which object gets passed into the VGA_adapter

```
//Ireen: these assign statements decide which object (pacman, the coins, or game won screen) gets drawn
  assign MUX_x =
   (whichObject == 3'b000) ? pacman_x :
   (whichObject == 3'b001) ? coin1_x :
   (whichObject == 3'b010) ? coin2_x :
   (whichObject == 3'b011) ? coin3_x :
   (whichObject == 3'b100) ? gameWon_x : 1'b0;

  assign MUX_y =
   (whichObject == 3'b000) ? pacman_y :
   (whichObject == 3'b001) ? coin1_y :
   (whichObject == 3'b010) ? coin2_y :
   (whichObject == 3'b011) ? coin3_y :
   (whichObject == 3'b100) ? gameWon_y : 1'b0;

  assign MUX_color =
   (whichObject == 3'b000) ? pacman_color :
   (whichObject == 3'b001) ? coin1_color :
   (whichObject == 3'b010) ? coin2_color :
   (whichObject == 3'b011) ? coin3_color :
   (whichObject == 3'b100) ? gameWon_color : 1'b0;

  assign MUX_write =
   (whichObject == 3'b000) ? pacman_write :
   (whichObject == 3'b001) ? coin1_write :
   (whichObject == 3'b010) ? coin2_write :
   (whichObject == 3'b011) ? coin3_write :
   (whichObject == 3'b100) ? gameWon_write : 1'b0;
```

Display Score on HEX0

This code instantiates the 7-segment decoder used to display the player's score on HEX0.

```
312     hex7seg H0 (score, HEX0);
```

Ghost Logic

The ghost has not yet been implemented into the main code, but the logic to make it work is already completed. This consists of two modules: one for moving the ghost, and another which checks collision between the ghost and Pacman.

In the ghost_mover module, we use counters (similar to the ones used in the upDn module) to increment or decrement the ghost's coordinates depending on its movement. Because the ghost moves along a predetermined path, its direction is controlled completely internally and will not be connected to the direction control logic seen above. After planning the path of the ghost, we took note of the coordinates where it would need to change direction and stored them in parameters. We then use if-statements to check if the ghost is at one of those positions and change its direction accordingly. It will always increment/decrement its coordinates depending on its current direction.

```
1  //This module is a counter which will increment the position of the ghost to move it along a
2  //pre-determined path
3
4  module ghost_mover(Clock, Resetn, currentX, currentY);
5     input Clock, Resetn;
6     output reg [7:0] currentX;
7     output reg [6:0] currentY;
8
9     reg moveUp, moveDown, moveLeft, moveRight;
10    reg cycle; //Controls which path ghost should take when reaching P1 (it can either move towards P2 or P7)
11
12    //Parameters hold turning points (the ghost should turn when it reaches this position)
13    parameter P1_X = 8'd152, P1_Y = 7'd70, P2_X = 8'd44, P2_Y = 7'd70, P3_X = 8'd43, P3_Y = 7'd4, P4_X = 8'd3, P4_Y = 
14              P5_X = 8'd3, P5_Y = 7'd112, P6_X = 8'd152, P6_Y = 7'd112, P7_X = 8'd152, P7_Y = 7'd4;
15
16    //Parameter for default position
17    parameter defaultX = 8'd151, defaultY = 7'd70;
18
19    wire [0:6] atP;
20
21    //Used for ModelSim testing to confirm that the code registers when the ghost hits a point where it needs to chang
22    //(same conditions as the if statements)
23    assign atP = {currentX == P1_X & currentY == P1_Y, currentX == P2_X & currentY == P2_Y, currentX == P3_X & current
24          currentX == P4_X & currentY == P4_Y, currentX == P5_X & currentY == P5_Y, currentX == P6_X & currentY == P6_
25          currentX == P7_X & currentY == P7_Y};
26
27    always@(posedge Clock)
28    begin
29       //Reset to default values
30       if (!Resetn)
31          begin
32             moveUp <= 0;
33             moveDown <= 0;
34             moveLeft <= 1;
35             moveLeft <= 1;
36             moveRight <= 0;
37
38             currentX <= defaultX;
39             currentY <= defaultY;
40             cycle <= 1;
41          end
42       else
43          begin
44          //Move the ghost in the direction it should be moving in
45          if (moveUp)
46             currentY <= currentY - 1;
47          else if (moveDown)
48             currentY <= currentY + 1;
49          else if (moveLeft)
50             currentX <= currentX - 1;
51          else if (moveRight)
52             currentX <= currentX + 1;
53
54          //Check if the ghost should change direction
55          //I use if statements over case statements because I want to check against two values and
56          //case statements can only be used to check against one value
57          if (currentX == P1_X & currentY == P1_Y)
58             begin
59             if (cycle)
60                begin
61                   moveUp <= 0;
62                   moveLeft <= 1;
63                   cycle <= 0;
64                end
65             else if (!cycle)
66                cycle <= 1;
67             end
68          else if (currentX == P2_X & currentY == P2_Y)
```

```
        else if (currentX == P2_X & currentY == P2_Y)
            begin
                moveLeft <= 0;
                moveUp <= 1;
            end
        else if (currentX == P3_X & currentY == P3_Y)
            begin
                moveUp <= 0;
                moveLeft <= 1;
            end
        else if (currentX == P4_X & currentY == P4_Y)
            begin
                moveLeft <= 0;
                moveDown <= 1;
            end
        else if (currentX == P5_X & currentY == P5_Y)
            begin
                moveDown <= 0;
                moveRight <= 1;
            end
        else if (currentX == P6_X & currentY == P6_Y)
            begin
                moveUp <= 1;
                moveRight <= 0;
            end
        else if (currentX == P7_X & currentY == P7_Y)
            begin
                moveUp <= 0;
                moveLeft <= 1;
            end
    end

  end

endmodule
```

```

//This module takes the coordinates of pacman and the ghost and it outputs 1 if they are colliding. If not colliding
//outputs 0
module ghost_pacman_collision_checker(pacmanX, pacmanY, ghostX, ghostY, result);
    input [7:0] pacmanX, ghostX;
    output [6:0] pacmanY, ghostY;
    output result;

    wire [7:0] pacmanX_1, ghostX_1;
    wire [6:0] pacmanY_1, ghostY_1;

    assign pacmanX_1 = pacmanX + 8'd7;
    assign ghostX_1 = ghostX + 8'd7;
    assign pacmanY_1 = pacmanY + 7'd7;
    assign ghostY_1 = ghostY + 7'd7;

    wire verticalOverlap, horizontalOverlap;

    assign verticalOverlap = (pacmanY_1 >= ghostY) & (pacmanY <= ghostY_1);
    assign horizontalOverlap = (pacmanX_1 >= ghostX) & (pacmanX <= ghostX_1);

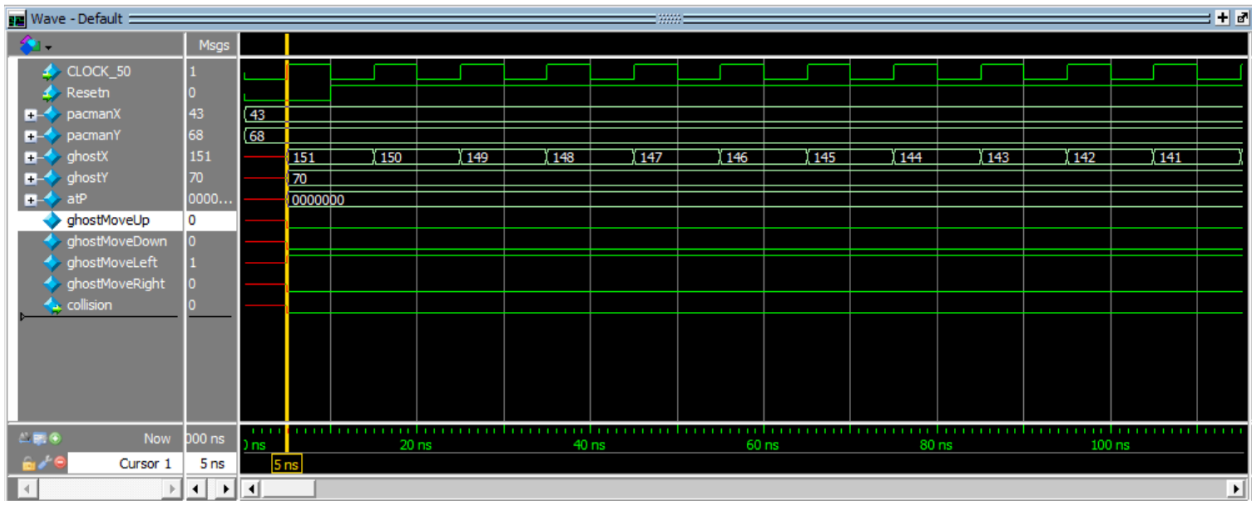
    assign result = verticalOverlap & horizontalOverlap;

endmodule
```

For the ModelSim, we created a module which instantiates both the ghost mover and collision checker. The module sets Pacman to his default coordinates in the middle of the maze. Also, because the simulation is very long (2000 ns), only portions of it are shown. This is because both modules have little input that can be changed and essentially work by themselves.

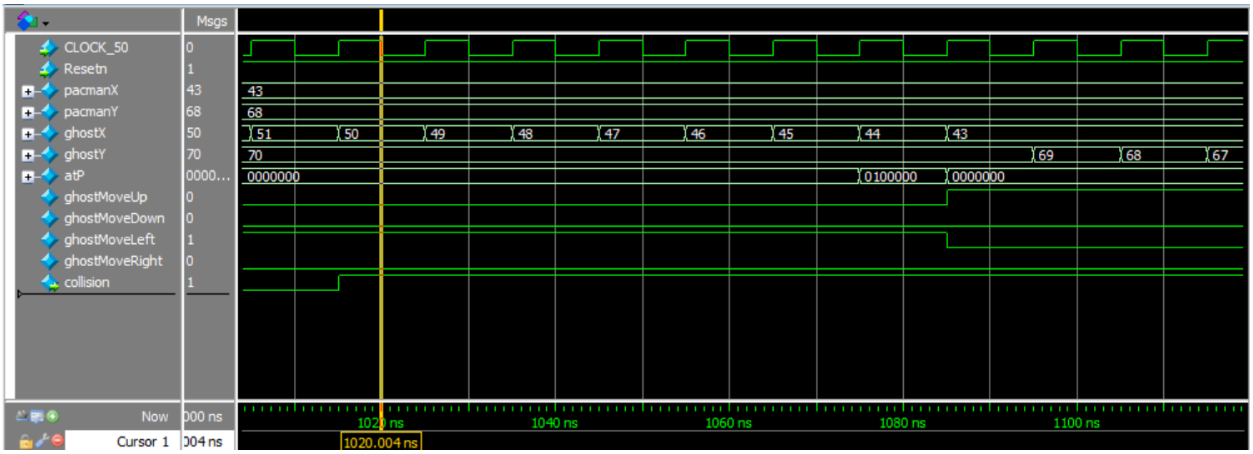
```
1  module main(Clock, Resetn, collision);
2     input Clock, Resetn;
3     output collision;
4
5     wire [7:0]ghostX, pacmanX;
6     wire [6:0]ghostY, pacmanY;
7
8     //Set pacmanX and Y to default positions
9     assign pacmanX = 8'd43;
10    assign pacmanY = 7'd68;
11
12    //instantiate ghost mover and collision modules
13    ghost_mover U1(Clock, Resetn, ghostX, ghostY);
14    ghost_pacman_collision_checker U2(pacmanX, pacmanY, ghostX, ghostY, collision);
15
16 endmodule
17
```

The below portion of the simulation shows, starting after Reset, that Pacman and the ghost are in their default positions. The ghost is moving left and there is no collision with Pacman.



After some time, the ghost gets close to Pacman. Even though Pacman is at (43, 70), the ghost collides with it at (50, 70) as the left edge of the ghost is touching the right edge of Pacman. This collision persists as the ghost continues to move left because the collision was designed to persist even when Pacman and the ghost are overlapping, unlike the other collision checker module, which only checks if the objects are touching, but not if they are overlapping.

When the ghost reaches (44, 70), we can see that the code registered that it is at a turning point (see that atP[1] changed from 0 to 1) and the ghost turns (moveLeft becomes 0 and moveUp becomes 1). We can see that after this point, the ghost begins moving up as its X coordinate is constant and its Y coordinate is decreasing.



When Y = 4, the ghost reaches another turning point (atP[2]) and turns so that it starts moving left.

